EXPERIMENT-72(HDOC)

C Programming — Enumerations

Program 1: Enumeration for the Days of the Week

Step 1: Understand the Problem Statement

- Input: None — the program does not read anything.
- Output: Every day from SUNDAY to SATURDAY with its integer value: `SUNDAY = 0, MONDAY = 1, TUESDAY = 2, WEDNESDAY = 3, THURSDAY = 4, FRIDAY = 5, SATURDAY = 6`.
- An enumeration is a user-defined type made of named integer constants, declared with the keyword `enum`. Here `enum Day { SUNDAY, MONDAY, ..., SATURDAY }` defines seven constants.
- When no values are written, the first name gets the value 0 and every following name gets the previous value plus 1, so SUNDAY is 0 and SATURDAY is 6. An enumeration value is an integer, so it can be printed with `%d` and used as a loop counter.
- The names are not stored by the enum itself, so an array of strings `dayNames[]` in the same order is used to print each name; the enum value is used as the index into this array.

Rule(s) used: enum Day { SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY } gives SUNDAY = 0 and each next constant = previous + 1, so the value of a day equals its position in the week counted from 0, and the loop `for (day = SUNDAY; day <= SATURDAY; day++)` visits all seven days.

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	No input	SUNDAY = 0, MONDAY = 1, TUESDAY = 2, WEDNESDAY = 3, THURSDAY = 4, FRIDAY = 5, SATURDAY = 6	SUNDAY = 0, MONDAY = 1, TUESDAY = 2, WEDNESDAY = 3, THURSDAY = 4, FRIDAY = 5, SATURDAY = 6	Pass

Step 3: Algorithm

1. Start the program.
2. Declare the enumeration `enum Day` with the constants SUNDAY to SATURDAY (values 0 to 6).
3. Call `printDays()`.
4. In `printDays()`, store the seven day names in the array `dayNames[]` in the order of the enumeration.
5. Set `day = SUNDAY` and repeat while `day <= SATURDAY`: print a comma and a space before every day except the first, then print `dayNames[day] = day`.

6. Increase day by 1 after each pass.
7. After the loop print a newline.
8. Stop the program.

Evaluation against Step 2 test case: the loop starts at SUNDAY (0) and ends at SATURDAY (6). It prints SUNDAY = 0, MONDAY = 1, TUESDAY = 2, WEDNESDAY = 3, THURSDAY = 4, FRIDAY = 5 and SATURDAY = 6, separated by commas — identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: C Program

File: hdoc73_1.c

```c
/* HDOC Day-73, Program 1: Enumeration for the days of the week.

   An enumeration (enum) is a user-defined type made of named integer
   constants. If no values are given, the first name gets 0 and every next
   name gets the previous value + 1, so SUNDAY = 0, MONDAY = 1, ...
   SATURDAY = 6. The program takes no input and prints every day together
   with its integer value.
*/

#include <stdio.h>

/* Enumeration for the days of the week */
enum Day { SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY };

/* Function prototype */
void printDays(void);

int main(void) {
    printDays();
    return 0;
}

/* Prints each day with its integer value */
void printDays(void) {
    /* names in the same order as the enumeration values 0 to 6 */
    const char *dayNames[] = { "SUNDAY", "MONDAY", "TUESDAY", "WEDNESDAY",
                               "THURSDAY", "FRIDAY", "SATURDAY" };
    enum Day day;

    for (day = SUNDAY; day <= SATURDAY; day++) {
        if (day != SUNDAY)
            printf(", ");
        printf("%s = %d", dayNames[day], day);   /* an enum value is an int */
    }
    printf("\n");
}
```

Step 5: Compile, Execute and Evaluate

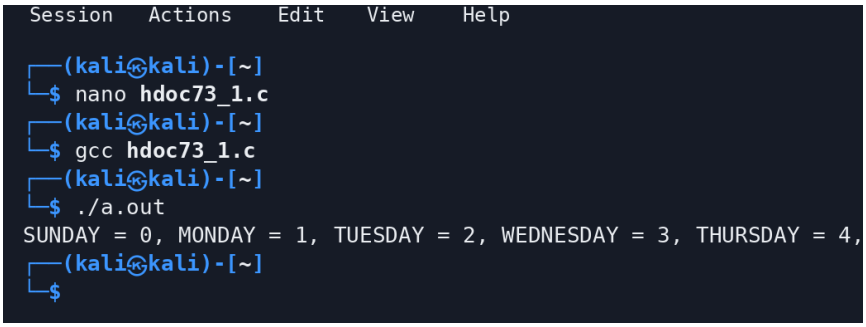
The program was compiled using the GCC compiler and executed from the Linux terminal using the following commands:

cc hdoc73_1.c	(compile)
./a.out	(execute)

All 1 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 6: Compilation and Execution Screenshot

Screenshot 1 of 1



Program 2: Enumeration for Traffic Lights

Step 1: Understand the Problem Statement

- Input: The traffic light colour — RED, YELLOW or GREEN (the letter case does not matter).
- Output: The action for that light: `Stop` for RED, `Wait` for YELLOW and `Go` for GREEN. Any other text is rejected with an error message.
- The enumeration `enum TrafficLight { RED, YELLOW, GREEN }` gives RED = 0, YELLOW = 1 and GREEN = 2.
- `parseLight()` converts the typed text to the matching enumeration value: the text is changed to upper case (`toupper()`) and compared with the three names using `strcmp()`. It returns 1 on success and 0 for an invalid colour.
- `showAction()` receives the enumeration value and uses a `switch` statement on it, printing Stop, Wait or Go for `case RED`, `case YELLOW` and `case GREEN`.

Rule(s) used: RED (0) → Stop, YELLOW (1) → Wait, GREEN (2) → Go. The text is converted to upper case before comparing, so "green", "Green" and "GREEN" all give the value 2.

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	`Input: GREEN`	Go	Go	Pass
2	`Input: RED`	Stop	Stop	Pass
3	`Input: YELLOW`	Wait	Wait	Pass
4	`Input: green`	Go	Go	Pass
5	`Input: PURPLE`	Invalid traffic light. Enter RED, YELLOW or GREEN.	Invalid traffic light. Enter RED, YELLOW or GREEN.	Pass

Step 3: Algorithm

1. Start the program.
2. Declare the enumeration `enum TrafficLight` with RED, YELLOW and GREEN (values 0, 1 and 2).
3. Read the colour into the character array `input`.

4. Call `parseLight(input, &light)`: copy the text in upper case and compare it with "RED", "YELLOW" and "GREEN" using `strcmp()`; store the matching enumeration constant in `light` and return 1, or return 0 if nothing matches.

5. If `parseLight()` returns 0 (or nothing was read), display `Invalid traffic light. Enter RED, YELLOW or GREEN.` and stop.

6. Call `showAction(light)`.

7. In `showAction()`: if `light` is RED print `Stop`, if it is YELLOW print `Wait`, if it is GREEN print `Go`.

8. Stop the program.

Evaluation against Step 2 test cases: the input GREEN is converted to the enumeration value 2, so the switch prints Go. RED (value 0) prints Stop and YELLOW (value 1) prints Wait. The lower-case input green becomes GREEN after conversion and also prints Go, while PURPLE matches none of the three names and is rejected with the error message — identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: C Program

File: hdoc73_2.c

```
/* HDOC Day-73, Program 2: Enumeration for traffic lights.

   enum TrafficLight { RED, YELLOW, GREEN } gives RED = 0, YELLOW = 1 and
   GREEN = 2. The colour typed by the user is converted to the matching
   enumeration value and a switch statement prints the action for it:
       RED -> Stop        YELLOW -> Wait        GREEN -> Go
*/

#include <stdio.h>
#include <string.h>
#include <ctype.h>

/* Enumeration for the traffic lights */
enum TrafficLight { RED, YELLOW, GREEN };

/* Function prototypes */
int  parseLight(const char *text, enum TrafficLight *light);
void showAction(enum TrafficLight light);

int main(void) {
    char input[20];
    enum TrafficLight light;

    printf("Enter traffic light (RED, YELLOW, GREEN): ");
    if (scanf("%19s", input) != 1 || !parseLight(input, &light)) {
        printf("Invalid traffic light. Enter RED, YELLOW or GREEN.\n");
        return 1;
    }

    showAction(light);
    return 0;
}

/* Converts the typed colour (any letter case) to the enumeration value.
   Returns 1 on success and 0 if the text is not a valid colour. */
int parseLight(const char *text, enum TrafficLight *light) {
    char upper[20];
    int i;
```

```
    for (i = 0; text[i] != '\0' && i < 19; i++)
        upper[i] = (char)toupper((unsigned char)text[i]);
    upper[i] = '\0';

    if (strcmp(upper, "RED") == 0)
        *light = RED;
    else if (strcmp(upper, "YELLOW") == 0)
        *light = YELLOW;
    else if (strcmp(upper, "GREEN") == 0)
        *light = GREEN;
    else
        return 0;
    return 1;
}

/* Prints Stop, Wait or Go according to the value of the enumeration */
void showAction(enum TrafficLight light) {
    switch (light) {
        case RED:
            printf("Stop\n");
            break;
        case YELLOW:
            printf("Wait\n");
            break;
        case GREEN:
            printf("Go\n");
            break;
    }
}
```

Step 5: Compile, Execute and Evaluate

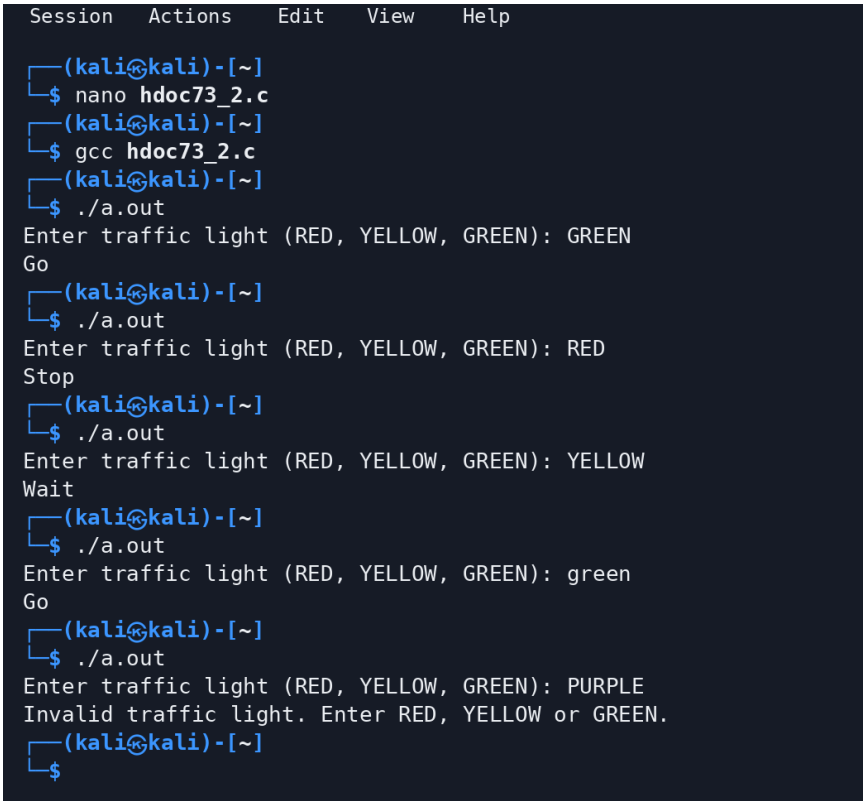
The program was compiled using the GCC compiler and executed from the Linux terminal using the following commands:

`cc hdoc73_2.c`	(compile)
`./a.out`	(execute)

All 5 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 6: Compilation and Execution Screenshot

Screenshot 1 of 1



Conclusion

In this experiment, enumerations were used to give meaningful names to integer constants. In the first program an enum for the days of the week was declared, whose names automatically take the values 0 (SUNDAY) to 6 (SATURDAY), and a loop printed every day with its integer value. In the second program an enum for traffic lights (RED = 0, YELLOW = 1, GREEN = 2) was used: the typed colour was converted to its enum value and a switch statement printed Stop, Wait or Go. Both programs were compiled with gcc and tested, including a lower-case and an invalid input, and every output matched the expected result.